

Cloud Computing



Cloud Computing

Conceptos Generales

- La Computación Cloud es un paradigma que posibilita el acceso ubicuo bajo demanda a servicios TIC accesibles a través de Internet.
- Modelos de Servicio: Cada modelo de servicio en la nube y método de implementación le aporta distintos niveles de control, flexibilidad y administración.
 - SaaS: El consumidor utiliza las aplicaciones del proveedor que son ejecutadas en una infraestructura cloud.
 - PaaS: El consumidor despliega aplicaciones tanto propias como adquiridas, desarrolladas usando entornos de programación soportados por el proveedor, en la infraestructura cloud de este.
 - IaaS: El consumidor aprovisiona recursos de computación (p.e. CPU, almacenamiento, red, ...) en los que ejecuta su software (incluidas aplicaciones y sistemas operativos).

Cloud Computing

¿Qué vamos a hacer?

- Crearemos un servicio web utilizando Flask y Gunicorn (Python)
Desplegaremos en un PaaS (<https://fly.io/> — <https://dashboard.render.com/>)

Cloud Computing

¿Qué necesitamos para trabajar?

- Cuenta de usuario en fly.io.

Interfaz de línea de comandos de Fly (fly CLI). ([descarga](#))

- Python
- PIP (administrador de paquetes)
- Descargar código de repositorio (<https://github.com/dssdunlp/cnpython-flyio.git>)

Cloud Computing

FLY.io (<https://fly.io/>)

Fly es una plataforma como servicio para ejecutar aplicaciones y bases de datos completas. Se presenta de manera muy similar a Heroku y se ofrece como alternativa ante el inminente cambio de políticas de este último.

Render (<https://dashboard.render.com/>)

Otra Plataforma como servicio como competencia de heroku presentando algunas ventajas como la simplicidad para desplegar aplicaciones dockerizadas.

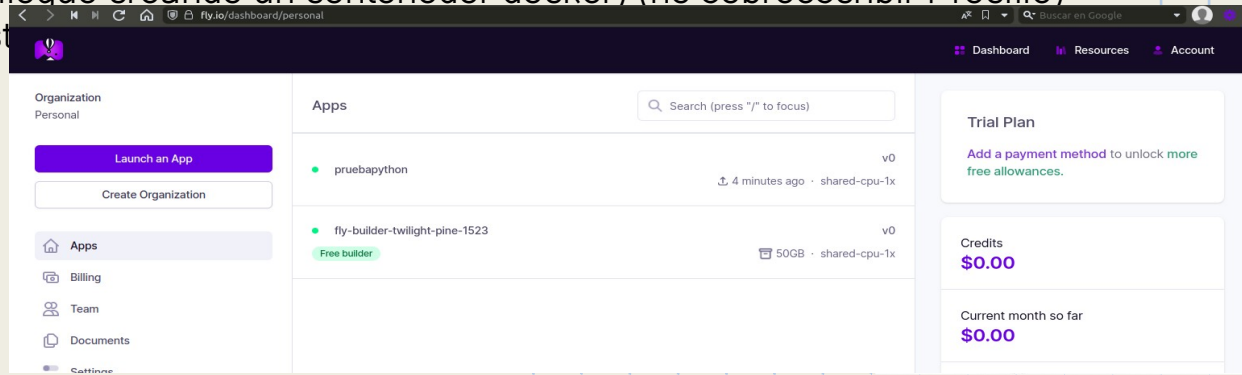
Cloud Computing

Creando nuestra Aplicación

- 1- Instalar y loguear en fly.io usando flyctl (cli)
- 2- git clone <https://github.com/dssdunlp/cnpython-flyio>
- 3- cd cnpython-flyio
- 3.1- En el archivo Procfile definimos como única línea : web: gunicorn index:app
- 4- flyctl launch (seguir los pasos del wizard)
- 5- flyctl deploy (se realiza el despliegue creando un contenedor docker) (no sobrescribir Procfile)
- 6- flyctl status (para conocer el est

Dashboard---

>



Cloud Computing

DOCKER:

La virtualización basada en contenedores, también llamada virtualización del sistema operativo, es una aproximación a la virtualización en la cual la capa de virtualización se ejecuta como una aplicación en el sistema operativo (OS). Con la virtualización basada en contenedores, no existe la sobrecarga asociada con tener a cada huésped ejecutando un sistema operativo completamente instalado.

Es una virtualización ligera, docker “enjaula procesos”, porque utiliza los mismos recursos del SO que se está ejecutando.

<https://www.docker.com>

<https://docs.docker.com>

Cloud Computing

DOCKER:

Imágenes y Contenedores

Una Imagen es un binario, una plantilla que permite desplegar un contenedor.

Las imágenes se descargan en capas (por decirlo de alguna manera), entonces cuando bajamos una imagen se van descargando otros componentes o software adicional necesario para correr la imagen.

Contenedor:

A diferencia de una máquina virtual que proporciona virtualización de hardware, un contenedor proporciona virtualización ligera a nivel de sistema operativo. Los contenedores comparten el núcleo del sistema huésped con otros contenedores.

Un contenedor, que se ejecuta en el sistema operativo host, es una unidad de software estándar que empaqueta código y todas sus dependencias, para que las aplicaciones se puedan ejecutar de forma rápida y fiable de un entorno a otro. Los contenedores no son persistentes y se activan desde imágenes.

Cloud Computing

DOCKER: comandos básicos para contenedores

Lifecycle

docker create crea un contenedor pero no lo comienza.

docker rename permite renombrar al contenedor.

docker run crea y comienza un contenedor en una operación.

docker rm borra un contenedor.

Iniciando y deteniendo

docker start comienza un contenedor si se cayó o salió.

docker stop detiene un contenedor.

docker restart detiene y comienza un contenedor.

docker pause pausa un contenedor corriendo, "lo congela".

docker unpause quita la pausa de un contenedor corriendo.

docker wait bloquea hasta que un contenedor corriendo se detiene.

docker kill envía una SIGKILL a un contenedor corriendo.

docker attach se conecta a un contenedor corriendo.

Cloud Computing

DOCKER: comandos básicos para contenedores

Info

docker ps muestra los contenedores corriendo.

docker logs obtiene logs de un container.

docker inspect observa toda la info en un contenedor.

docker events obtiene eventos de un contenedor.

docker port muestra el puerto publico de un contenedor.

docker top muestra los procesos corriendo en un contenedor.

docker stats muestra las estadísticas de recursos usados por contenedor.

docker diff muestra los archivos cambiados en el FS del contenedor.

docker ps -a muestra todos los contenedores corriendo y detenidos.

docker stats --all muestra una lista de los contenedores corriendo.

Cloud Computing

DOCKER: comandos básicos para imágenes

Lifecycle

docker images muestra todas las imagenes

docker build crea imagen de un Dockerfile.

docker rmi remueve una imagen.

Info

docker history muestra el historial de una imagen.

docker tag taggea una imagen a un nombre asignado.

Cloud Computing

DOCKER: primeros pasos (local)

1- Descargar una imagen.

```
docker pull dssdunlp/renderprueba:01
```

2- Correr un contenedor.

```
docker run -it dssdunlp/renderprueba:01
```

Probamos el servicio (<http://xxx:yyy/hello?name=AAA>)

3- Detener un contenedor.

```
Docker stop <nombre contenedor>
```

4- Asignar un volumen a un contenedor.

```
docker run -it -v <carpetahost>:<carpetacontenedor> dssdunlp/renderprueba:01
```

5- Eliminar un contenedor.

```
docker rm <nombre contenedor>
```

6- Eliminar una imagen.

```
docker rmi <nombre contenedor>
```

Cloud Computing

DOCKER: desplegando en un PaaS (Render)

En el caso de Render vamos a necesitar un archivo Dockerfile alojado en un repositorio para desplegar la app.

Dockerfile: Un Dockerfile es un documento de texto simple que incluye las instrucciones los procesos necesarios para la creación de una nueva imagen.

A partir de un archivo Dockerfile podemos crear una imagen y compartirla en forma binaria o compartir el archivo Dockerfile (liviano) para crear nuestras propias imágenes.

Cloud Computing

DOCKER: desplegando en un PaaS (Render)

```
#Create a ubuntu base image with python 3 installed.  
FROM python:3.8
```

```
#Set the working directory  
WORKDIR /
```

```
#copy all the files  
COPY . .
```

```
#Install the dependencies  
RUN apt-get -y update  
RUN apt-get update && apt-get install -y python3 python3-pip  
RUN pip3 install -r requirements.txt
```

Cloud Computing

DOCKER: desplegando en un PaaS (Render)

Comandos Dockerfile

FROM — Especifica la base para la imagen.

ENV — Establece una variable de entorno persistente.

RUN — Ejecuta el comando especificado. Se usa para instalar paquetes en el contenedor.

COPY — Copia archivos y directorios al contenedor.

ADD — Lo mismo que COPY pero con la funcionalidad añadida de descomprimir archivos .tar y la capacidad de añadir archivos vía URL.

WORKDIR — Indica el directorio sobre el que se van a aplicar las instrucciones siguientes.

CMD — Especifica el comando y argumentos que se van a pasar al contenedor una vez instanciado.

EXPOSE — Indica que puerto del contenedor se debe exponer al ejecutarlo.

VOLUME — Crea un directorio sobre el que se va a montar un volumen para persistir datos más allá de la vida del contenedor.

```
docker build -f <nombreDockerfile> -t <nombreImagen>:<tag> "."
```

Cloud Computing

DOCKER: desplegando en un PaaS (Render)

Comandos Dockerfile

FROM — Especifica la base para la imagen.

ENV — Establece una variable de entorno persistente.

RUN — Ejecuta el comando especificado. Se usa para instalar paquetes en el contenedor.

COPY — Copia archivos y directorios al contenedor.

ADD — Lo mismo que COPY pero con la funcionalidad añadida de descomprimir archivos .tar y la capacidad de añadir archivos vía URL.

WORKDIR — Indica el directorio sobre el que se van a aplicar las instrucciones siguientes.

CMD — Especifica el comando y argumentos que se van a pasar al contenedor una vez instanciado.

EXPOSE — Indica que puerto del contenedor se debe exponer al ejecutarlo.

VOLUME — Crea un directorio sobre el que se va a montar un volumen para persistir datos más allá de la vida del contenedor.

```
docker build -f <nombreDockerfile> -t <nombreImagen>:<tag> "."
```


Cloud Computing

DOCKER: desplegando en un PaaS (Render)

```
#Create a ubuntu base image with python 3 installed.  
FROM python:3.8
```

```
#Set the working directory  
WORKDIR /
```

```
#copy all the files  
COPY . .
```

```
#Install the dependencies  
RUN apt-get -y update  
RUN apt-get update && apt-get install -y python3 python3-pip  
RUN pip3 install -r requirements.txt
```

Cloud Computing

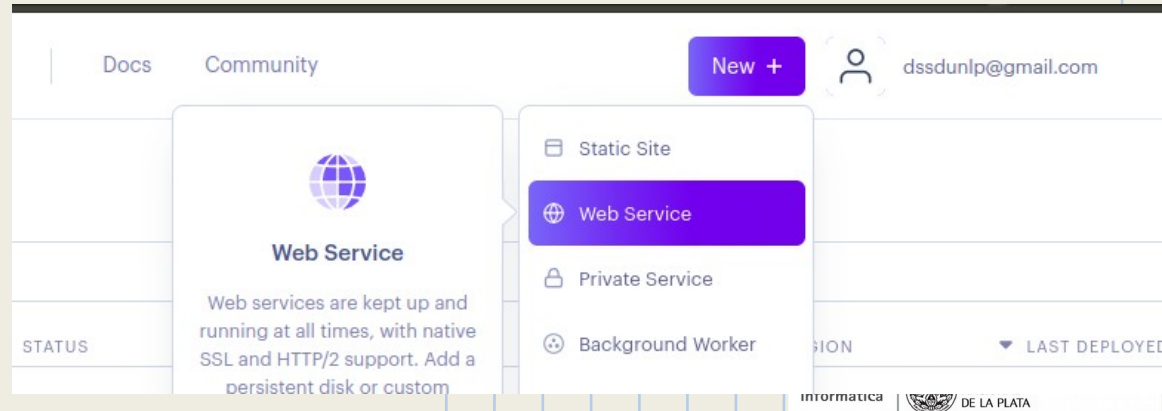
DOCKER: desplegando en un PaaS (Render)

Una vez creado el archivo Dockerfile junto con todo el código necesario lo sincronizamos en un repositorio github/gitlab (ejemplo:

<https://github.com/dssdunlp/renderprueba>)

1- Login en <https://dashboard.render.com>

2- Creamos un web service.



Cloud Computing

DOCKER: desplegando en un PaaS (Render)

3- Seleccionamos el repositorio con todo el código necesario incluido el Dockerfile.

Create a new **Web Service**

Connect your Git repository or use an existing public repository URL.

Connect a repository

 dssdunlp / renderprueba • 8 minutes ago

Connect

 dssdunlp / cnpython-flyio • 2 days ago

Connect

Cloud Computing

DOCKER: desplegando en un PaaS (Render)

4- Damos nombre a la app y seteamos parámetros de configuración guardando los cambios.

You are deploying a web service for [dssdunlp/renderprueba](#).

You seem to be using **Docker**, so we've autofilled some fields accordingly. Make sure the values look right to you!

Name

A unique name for your web service.

renderprueba2

Root Directory

Render automatically deploys your service when files change inside the [root directory](#). Render runs commands

e.g. src

Docker Command

Add an optional command to override the Docker **CMD** for this service. This will also override the **ENTRYPOINT** if defined in your Dockerfile. Examples: `./start.sh --type=worker` or `./bin/bash -c cd /some/dir && ./start.sh`

```
gunicorn --bind 0.0.0.0:5000 main:app
```

Please enter your payment information t

Current

Free

RAM 512 MB
CPU shared

\$0/month

Cloud Computing

DOCKER: desplegando en un PaaS (Render)

5- Se despliega nuestra app y se nos informa la url.

The screenshot shows the Render dashboard for a web service named "renderprueba2". At the top, it indicates the service is using Docker and is on the Free Plan. The repository is "dssdunlp/renderprueba" and the branch is "master". A "Connect" button and a "Manual Deploy" button are visible. Below this, the URL "https://renderprueba2.onrender.com" is shown. On the left, a sidebar lists navigation options: Events, Logs, Disks, Environment, Shell, PRs, and Jobs. The main content area shows a message about slow builds and a log entry for a deployment on September 22, 2022, at 10:57 AM, which is currently "In progress". The log entry text is "cbad07c actualizacion para TP". At the bottom, there is a search bar for logs and buttons for "Maximize" and "Scroll to top".

WEB SERVICE

renderprueba2 Docker Free Plan dssdunlp/renderprueba master

<https://renderprueba2.onrender.com>

Connect Manual Deploy

Events

Builds too slow? [Upgrade to a paid plan](#) to go faster. Learn more about [free plan limits](#).

Logs

September 22, 2022 at 10:57 AM *** In progress

Disks

Environment

Shell

Search Maximize Scroll to top

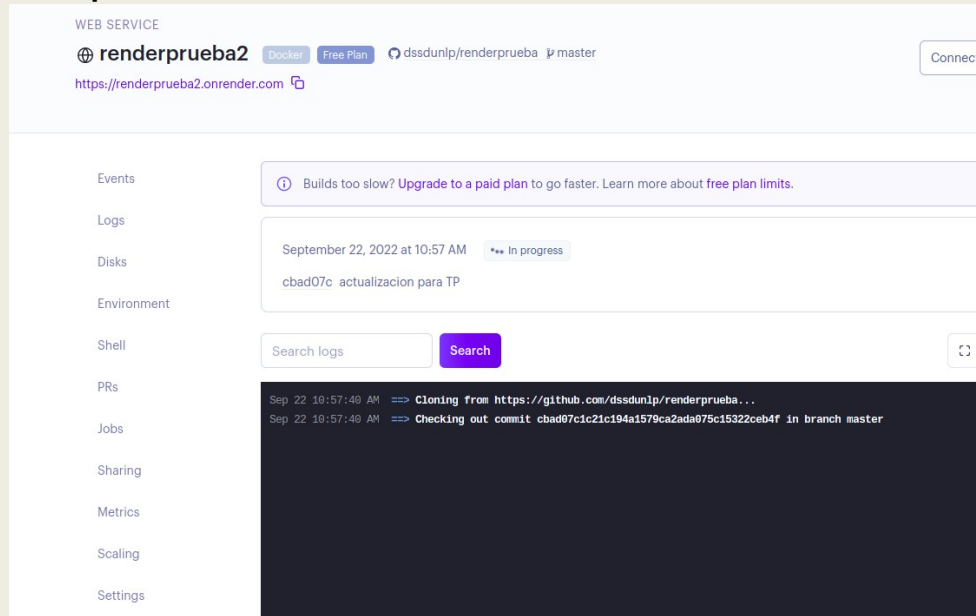
PRs

Jobs

Cloud Computing

DOCKER: desplegando en un PaaS (Render)

5- En el menú de la izquierda podemos utilizar diferentes herramientas sobre nuestra app.



The screenshot shows the Render dashboard for a service named 'renderprueba2'. The interface includes a sidebar with navigation options: Events, Logs, Disks, Environment, Shell, PRs, Jobs, Sharing, Metrics, Scaling, and Settings. The main content area displays the service status as 'Free Plan' and 'dssdunlp/renderprueba2 master'. A notification banner indicates that builds are slow and suggests upgrading to a paid plan. Below this, a log entry for September 22, 2022, at 10:57 AM shows the build process in progress. The log text includes 'Cloning from https://github.com/dssdunlp/renderprueba...' and 'Checking out commit cbad07c1c21c194a1579ca2ada075c15322ceb4f in branch master'.

Cloud Computing

Alternativas

IBM Cloud (<https://cloud.ibm.com/login>)

CLEVER CLOUD (<https://www.clever-cloud.com>)

OPEN SHIFT (<https://www.openshift.com>)

Platform.sh (<https://auth.api.platform.sh>)

- IBM Cloud: es la PaaS de IBM que permite realizar pruebas de forma gratuita durante un mes. Presenta una interfaz web para su gestión o por medio de una CLI.
- OPEN SHIFT: Características más relevantes de Open Shift. Permite el desarrollo rápido de aplicaciones escalables y alojadas en un cloud público. Soporte de la Comunidad(Free Plan); Red Hat(Premium Plan). Se ejecuta en el cloud público. Está especialmente pensado para startups, desarrolladores y pequeñas empresas.
- Platform.sh: muy similar a heroku.

Cloud Computing

Herramientas adicionales

- GIT: Git es un sistema gratuito y de código abierto, de control de versiones diseñado para manejar desde pequeños a grandes proyectos con rapidez y eficiencia. En nuestro caso se utilizará para clonar el código de nuestro cartridge y sincronizarlo (actualizarlo) en la nube.

```
git clone ssh:.....  
git config --global user.email "xxx@gmail.com" (por única  
vez)  
git add .  
git commit -a -m 'prueba de actualizacion'  
git push
```


Cloud Computing

Bibliografía / Tutoriales / Links de Interés

<https://render.com/docs>

<https://fly.io/docs/>

<https://docs.docker.com>

<https://hub.docker.com>

Alternativa como PaaS

<https://auth.api.platform.sh>